



华南理工大学

实验报告

课程名称： 软件分析设计与建模

学生姓名： 马启凡、叶炳良、周子栋、庄子杰

学生学号： 202430551135 (马)
202430552848 (叶)
202430553005 (周)
202430552224 (庄)

学生专业： 软件工程 1 班

开课学期： 2025-2026 第二学期

软件学院
2026 年 4 月

目录

第一章 初始阶段.....	3
1.1 系统背景.....	3
1.2 参与者识别.....	5
1.2.1 客户.....	5
1.2.2 银行核心系统.....	6
1.2.3 ATM 设备.....	6
1.2.4 管理员（可选参与者）.....	6
1.3 系统边界与功能范围.....	7
1.3.1 术语表（Glossary）.....	7
1.3.2 系统功能边界说明.....	10
1.4 用例图.....	11
1.5 核心用例规约.....	13
1.5.1 用例一：登录认证.....	13
1.5.2 用例二：查询余额.....	14
1.5.3 用例三：取款.....	15
1.5.4 用例四：转账.....	16
1.6 本章小结与补充说明.....	18
第二章 细化迭代 1.....	19
2.1 本次迭代目标与实现范围.....	20
2.2 显示模块的第一次迭代职责.....	21
2.3 领域模型.....	22
2.4 系统顺序图.....	23
2.5 操作契约.....	26
2.6 UML 包图.....	28
2.7 UML 交互图.....	29

2.8 UML 类图.....	30
2.9 本次迭代实现与验收说明.....	31
2.10 本章小结	32

迭代整体计划

提交章节	具体内容	提交时间
第一章 初始（Inception）	Use-Case Model, Vision, Supplementary Specification, Glossary, Iteration Plan	第5周周五晚
第二章 细化迭代1	Domain Models, System Sequence Diagrams, Operation Contracts, UML Package Diagrams, UML Interaction Diagrams, UML Class Diagrams	第9周周五晚
第三章 细化迭代2	Applying GoF Design Patterns	第10周周五晚
第四章 细化迭代3	UML State Machine Diagrams and Modeling, refine the above models, 汇总完整的文档	第14周周五晚

第一章 初始阶段

1.1 系统背景

ATM 自动取款机系统是银行面向客户提供自助金融服务的重要终端。客户通过 ATM 可以在非柜台场景下完成取款、存款、查询余额、转账、修改密码、打印凭条和退卡等业务，从而缓解人工柜台压力，提高服务效率，并改善客户在常规营业时间之外的自助服务体验。

本课程作业以虚拟 ATM 系统为研究对象，结合《软件分析设计与建模实验要求》和既有 UML 作业方案，在“先分析建模、后迭代细化”的思路下推进。项目采用前后端分离架构：前端使用 Vue 3、Vite、Vue Router、Pinia、Axios，后端使用 Java 17、Spring Boot、Spring Web、Validation、MyBatis，数据存储采用 MySQL，建模与接口说明采用 Markdown、Mermaid 和 OpenAPI。

结合《UML 和模式应用》第二部分“初始阶段”的观点，初始阶段并不是一次性定义全部需求，也不是在项目一开始就形成可靠预算和完整计划；它更像是一个较短的起始步骤，用来建立项目共同设想、划定基本范围、识别关键业务案例、发现重要风险，并判断项目是否值得继续深入。对 ATM 系统而言，本章的任务不是把所有功能都详细写完，而是在课程要求允许的粒度下，把后续细化迭代真正需要的“骨架”先搭起来。

按照课本关于初始阶段的说明，本阶段关注的是“建立共同设想、形成合理范围、抓住关键用例与关键约束”，而不是提前穷尽全部需求。对于 ATM 课程项目，这意味着应优先识别登录认证、余额查询、取款、转账等对架构和业务最关键的场景，并为后续细化迭代保留扩展空间。

本系统的建设目标包括：建立一个可演示的 ATM 业务原型系统；支持 ATM 核心业务流程的分析与建模；为后续三次细化迭代提供统一的需求基础；保证 UML 模型、接口设计与后续代码实现能够相互对应。同时，依据实验要求，系统将在完整业务语义下展开分析，但在后续实现中优先围绕显示模块及其与认证、账户、交易服务之间的交互关系进行深化。

结合课本对“初始阶段制品”的描述和课程实验的分阶段交付要求，本章形成的主要成果可概括为下表。它们共同承担了“建立项目共同语言、明确边界、抓住关键场景、暴露主要风险”的职责，并将作为第二章及以后各迭代文档的输入。

制品	在本文档中的对应	作用说明
设想与系统背景	1.1 系统背景	用于建立项目共同设想，说明 ATM 系统建设动因、价值目标与课程交付定位。
用例模型	1.4 用例图、1.5 核心用例规约	用于识别主要参与者及用户目标，并优先细化对架构和业务影响最大的代表性用例。
补充性规格说明	1.3 系统边界与功能范围	记录用例之外的重要约束，尤其是 FURPS+ 视角下的初始非功能需求与接口约束。
词汇与领域术语	1.1、1.2、1.3 相关术语	保证“账户、会话、交易流水、银行核心系统”等关键

制品	在本文档中的对应	作用说明
	语说明	概念在小组内部表达一致。
风险与初始迭代关注点	1.3、1.5 分析说明	聚焦认证、交易一致性、设备故障、网络异常等高风险点，为后续细化迭代排序。
初步计划与分阶段交付依据	1.1、1.6	与实验要求中的“按 UP 过程分阶段完善文档”保持一致，为后续细化迭代留出扩展空间。

根据既有作业方案，系统后续将围绕插卡登录、密码验证、查询余额、取款、存款、转账、修改密码、打印凭条和退卡等核心功能逐步展开。本章作为初始阶段文档，不追求一次性完成全部功能说明，而是优先完成系统背景说明、参与者识别、系统边界界定、总体用例视图及关键用例规约，为后续分析类图、活动图、顺序图和设计模型奠定基础。

1.2 参与者识别

在用例建模中，参与者是位于“被讨论系统”之外、与系统发生交互并具有明确目标或职责的外部实体。它可以是人、外部系统、组织，甚至是对系统施加约束的运行环境。因而，识别参与者的关键不在于“谁出现在图上”，而在于“谁对系统提出目标、请求服务、提供支撑或触发约束”。结合 ATM 业务场景和课程作业范围，本系统的主要参与者如下。

1.2.1 客户

客户是 ATM 系统的主要直接参与者，也是最典型的“用户目标驱动型”参与者。客户通过插卡、输入密码、选择菜单和确认操作，与 ATM 完成交互，其目标通常不是“操作某个按钮”，而是完成诸如取款、查询余额、转账、修改密码和打印凭条等具体业务。按照用例思维，客户关注的是“能否顺利完成目标”，因此本章后续用例规约也将以客户目标为中心组织。

- 登录认证与建立会话。
- 查询当前账户余额与基础账户信息。

- 执行取款、存款、转账等交易。
- 修改密码、打印凭条、退卡并安全结束会话。

1.2.2 银行核心系统

银行核心系统是 ATM 的外部支撑参与者，本身不直接操作 ATM 界面，但会对认证、账户与交易处理结果产生决定性影响。它承担的职责包括银行卡与密码校验、账户信息查询、余额核对、交易处理确认、交易流水留痕等。由于它位于 ATM 系统边界之外，因此在分析中应被视为外部系统接口，而不是 ATM 内部对象。

- 校验银行卡状态、账户状态及密码合法性。
- 提供账户余额、账户基本信息及目标账户校验能力。
- 支持取款、存款、转账等交易的提交与结果返回。
- 记录关键交易流水，为审计和对账提供依据。

1.2.3 ATM 设备

严格来说，ATM 设备本身更接近系统运行环境与边界资源，而非典型的“外部业务参与者”；但在 ATM 场景中，读卡器、键盘、屏幕、吐钞模块和打印模块都对业务执行结果有直接影响，因此在课程建模中仍有必要对其进行显式说明。将其单独识别出来，有助于后续在顺序图、状态图或设计模型中划分“界面控制、设备控制、业务服务”的职责边界。

- 接收客户输入，展示菜单、提示与结果信息。
- 执行读卡、吐钞、打印凭条和退卡等设备动作。
- 反馈设备状态，如现金不足、打印机异常、读卡失败等。

1.2.4 管理员（可选参与者）

管理员不是本次初始阶段的核心参与者，但在扩展场景中是不可忽视的运维角色。其职责包括设备状态检查、现金补充、故障处理、日志查看和基础配置维护。考虑到课程项目后续可能扩展到系统管理、异常处理与部署演示，本章保留管理员作为可扩展角色，但暂不把它列为关键用例规约对象。

综上，本项目可将客户视为主要参与者，将银行核心系统视为外部支撑参与者，将 ATM 设备视为重要的边界与运行资源，将管理员视为可选的维护参与者。这样的识别方式既符合 ATM 业务实际，也有利于后续建模时明确“系统内部职责”和“系统外部依赖”的分界。

1.3 系统边界与功能范围

1.3.1 术语表（Glossary）

为保证小组成员在 ATM 系统分析、建模与后续实现过程中对关键概念的理解保持一致，现对本文涉及的核心术语说明如下。

术语	英文/对应表达	定义说明
ATM 系统	ATM System	指本项目所讨论的自动取款机业务系统，负责会话控制、认证协调、账户查询、交易处理、设备控制及结果反馈。
客户	Customer	指使用 ATM 设备办理业务的直接用户，是系统的主要参与者。
银行核心系统	Bank Core System	指位于 ATM 系统边界之外的外部支撑系统，负责账户校验、密码验证、余额查询、交易处理与流水记录等。
ATM 设备	ATM Device	指 ATM 的物理设备及其外围部件，如读卡器、键盘、显示屏、吐钞模块、凭条打印模块等，是系统运行的重要边界资源。
管理员	Administrator	指负责 ATM 设备维护、现金补充、故障处理和日志查看的运维角色，在本阶段属于可选参与者。
会话	Session	指客户从登录认证成功开始，到退卡或超时结束之间形成的一次有效交互过程。会话存在期间客户才能访问相关业务功能。
登录认证	Authentication / Login	指客户通过插卡和输入密码完成身份校验的过程，其结果决定是否允许进入系统主菜单。

术语	英文/对应表达	定义说明
身份验证	Identity Verification	指系统对客户银行卡状态、账户状态及密码合法性进行检查与确认的行为，是登录成功的前提。
银行卡	Bank Card	指客户用于在 ATM 上发起身份认证和办理业务的实体介质，系统需识别其卡号、状态及可用性。
密码	PIN / Password	指客户登录 ATM 时输入的敏感认证信息，用于证明身份合法性，系统应采用遮蔽方式显示并限制重试次数。
主菜单	Main Menu	指客户登录成功后看到的业务导航界面，用于选择查询余额、取款、转账、修改密码等功能。
账户	Account	指客户在银行中的资金记录实体，是余额查询、取款、存款、转账等业务操作的对象。
账户状态	Account Status	指账户当前的可用情况，如正常、冻结、停用等，系统需依据账户状态决定是否允许业务继续执行。
余额	Balance	指账户当前可用于查询和交易的金额信息，是 ATM 系统的重要业务数据。
查询余额	Balance Inquiry	指客户在有效会话中查看当前登录账户可用余额的业务操作，属于只读型业务。
取款	Withdrawal	指客户通过 ATM 从当前账户提取现金的业务过程，涉及金额校验、余额检查、扣款、吐钞和流水记录。
存款	Deposit	指客户通过 ATM 向当前账户存入资金的业务过程，在完整系统中通常涉及金额确认、入账与流水记录。
转账	Transfer	指客户将当前账户中的指定金额转入目标账户的业务过程，要求保证转出与转入的一致性。
修改密码	Change Password	指客户在登录后修改账户认证密码的业务操

术语	英文/对应表达	定义说明
		作，需满足身份已验证和规则校验。
打印凭条	Print Receipt	指在业务完成后由 ATM 输出业务结果凭证的操作，用于向客户提供交易记录证明。
退卡	Card Eject / Return Card	指 ATM 在业务结束、认证失败或异常终止时将银行卡退出并结束当前会话的操作。
交易	Transaction	指任何会导致账户状态变化或产生业务记录的操作，如取款、存款、转账等。
交易流水	Transaction Record / Transaction Log	指系统对重要交易过程与结果形成的记录，用于审计、查询、对账和异常追踪。
交易编号	Transaction ID	指系统为一次交易生成的唯一标识，用于后续查询、审计和问题定位。
金额规则	Amount Rules	指系统对交易金额设置的业务限制，如最低金额、最高金额、步长单位等。
一致性	Consistency	指交易处理过程中系统必须避免出现部分成功、部分失败的账务异常状态，例如“已扣款但未吐钞且无记录”。
异常回滚	Rollback	指在交易过程中发生错误时，系统将未完成或不一致的业务状态恢复到安全状态的处理机制。
会话超时	Session Timeout	指客户在一定时间内无操作时，系统自动结束当前会话并返回安全状态的机制。
设备状态反馈	Device Status Feedback	指 ATM 对设备运行情况的提示，如现金不足、打印机异常、读卡失败等。
外部接口	External Interface	指 ATM 系统与银行核心系统之间进行认证、查询和交易通信的接口边界。
系统边界	System Boundary	指本项目分析对象所涵盖的范围，用于明确哪些功能属于 ATM 系统内部，哪些内容属于外部依赖。
补充性规格说	Supplementary	指对用例之外的重要约束进行说明的文档内容，通常包括非功能需求、接口约束和业务规

术语	英文/对应表达	定义说明
明	Specification	则等。
FURPS+	FURPS+	一种常用的需求分类方法，用于从功能性、可用性、可靠性、性能、可支持性及附加约束等方面补充描述系统需求。

1.3.2 系统功能边界说明

系统边界决定了分析对象是什么，也决定了哪些问题应该在本项目中回答、哪些问题应被保留到系统外部。结合 ATM 场景和已有作业方案，本文将被讨论系统定义为：客户通过 ATM 前端界面发起操作，ATM 系统负责会话控制、认证协调、账户与交易业务处理、设备动作控制以及结果反馈，并通过后台服务与数据库完成业务数据读写和外部校验。银行核心系统、真实硬件驱动协议及跨行网络不属于 ATM 系统内部实现。

从“范围而非穷尽需求”的角度看，初始阶段更强调先抓住高价值、高风险和高架构影响的问题。因此，本项目把以下内容纳入当前分析范围：身份认证（插卡、输入密码、登录验证、退卡）；账户业务（查询余额、查询账户信息、修改密码）；交易业务（取款、存款、转账）；辅助业务（打印凭条、查询交易流水、设备状态反馈）。这些内容覆盖了 ATM 系统的主要用户目标，也与后续三次细化迭代的功能划分保持一致。

与此同时，本课程项目有意不在初始阶段深入以下内容：真实银行级密码加密体系、跨行清算与结算细节、复杂风控与反欺诈策略、生产级设备驱动和硬件通信协议、严格意义上的高可用部署体系以及法规合规模块的完整实现。对这些内容，本章只在需要时以约束、风险或接口假设的形式加以说明，而不将其视为当前阶段必须完成的内部设计任务。

结合课本中对“补充性规格说明”和 FURPS+ 分类方法的介绍，ATM 系统在初始阶段除功能范围之外，还应同步识别若干关键的非功能需求和约束。这些内容虽然未必全部写入某个单一用例，却会显著影响后续架构选择、界面设计和异常处理方案。

维度	本项目在初始阶段的关注点
----	--------------

功能性 (F)	支持插卡登录、身份验证、查询余额、取款、存款、转账、修改密码、打印凭条、退卡等核心功能；其中登录、查询余额、取款和转账作为本阶段重点用例。
可用性 (U)	界面流程应简洁、菜单层级清晰、提示语言明确；错误信息要可理解且可恢复，便于非技术用户在 ATM 场景下独立完成操作。
可靠性 (R)	避免出现“扣款成功但吐钞失败无记录”之类的不一致状态；重要交易必须记录流水，密码重试、会话超时和异常回滚策略需在分析阶段先被识别。
性能 (P)	余额查询与菜单切换需保持良好响应；交易请求在后台服务可用时应在可接受时间内完成，并在网络异常时快速反馈而非长时间无响应。
可支持性 (S)	前后端分离、模块职责清晰、接口可测试；建模文档与实现结构相互对应，便于后续细化迭代和团队分工。
“+”约束	包括与银行核心系统的接口约束、ATM 设备能力约束、交易金额规则、日志审计、部署与实现技术约束等。

在此基础上，本项目还识别出若干初始风险：其一，登录认证、会话控制和交易处理都依赖外部银行核心系统，网络波动或接口不可用会直接影响客户体验；其二，取款、转账等交易具有一致性要求，必须避免“部分成功、部分失败”造成的账务错误；其三，ATM 设备状态（读卡、吐钞、打印）与业务结果存在联动，设备异常可能放大业务风险；其四，课程项目虽不追求真实银行级复杂度，但关键业务规则如金额限制、密码重试、超时退出、凭条策略等仍需尽早明确，以便后续设计不会反复返工。

课本强调：初始阶段不应试图定义大部分需求，也不应期待在此阶段得到可靠预算和完整计划。对 ATM 课程项目而言，更合理的做法是先明确系统边界和关键约束，再围绕高价值、高风险用例推进细化迭代。

1.4 用例图

用例模型的核心是文本形式的用例，而不是图本身；用例图更适合充当语境图和索引视图，帮助团队快速看到“谁与系统交互”和“系统对外提供了哪些主要能力”。因此，下图的作用不是替代用例说明，而是在初始阶段给出 ATM 系统的总体业务轮廓。

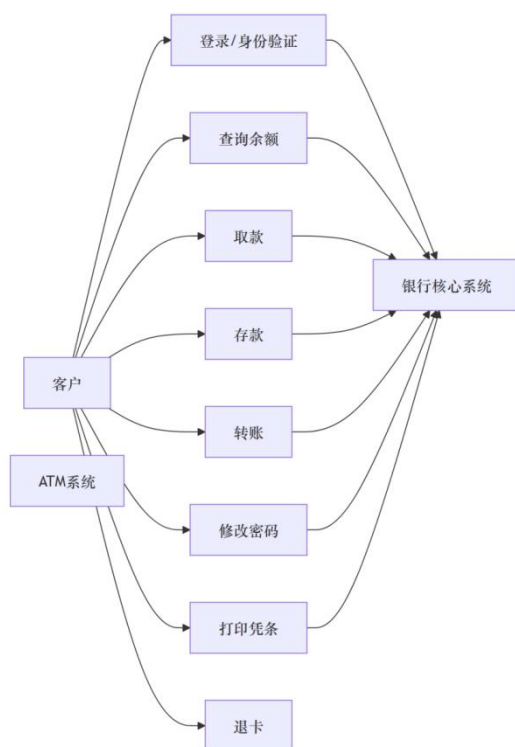


图 1-1 ATM 系统初始阶段总体用例关系图

由图可见，客户是触发全部核心业务的直接参与者，银行核心系统为身份验证、账户查询和交易处理提供外部支撑，而 ATM 系统则作为客户目标的承载者和协调者。图中“登录/身份验证、查询余额、取款、存款、转账、修改密码、打印凭条、退卡”等用例基本覆盖了当前阶段的主要用户目标。

需要强调的是，初始阶段用例图重在列出关键用例名称和参与者关系，帮助团队形成共同认识；真正用于后续分析和设计的关键输入仍然是用例文本本身。按照课本中“初始阶段只需较详细编写 10%~20% 关键用例”的思路，本章将在用例图的基础上选择代表性且具有架构意义的用例做相对详细的规约，而不是把所有用例都写成同样的细粒度。

1.5 核心用例规约

结合初始化阶段的任务要求、本项目已有方案以及课本关于“在初始阶段优先详细描述高风险、高价值、具有重要架构意义的用例”的建议，本文选择“登录认证”“查询余额”“取款”“转账”四个代表性用例进行规约说明。它们分别覆盖认证类、查询类、交易类和跨账户交易类场景，足以支撑后续分析类图、活动图和顺序图的细化。对于存款、修改密码、打印凭条等其他用例，将在后续细化迭代中按同样方法继续补充。

1.5.1 用例一：登录认证

- **用例名称：**登录认证
- **用例编号：**SUC001
- **主要参与者：**客户
- **目标：**建立合法有效的 ATM 会话，使客户进入主菜单并获得后续业务访问资格。
- **触发条件：**客户插入银行卡，开始身份验证。
- **前置条件：**ATM 设备处于待机状态；银行卡物理可读；系统与后台认证服务处于可用状态。
- **后置条件：**成功时建立当前会话并进入主菜单；失败时系统保持安全状态，必要时结束会话并退卡。
- **用例简述：**客户通过插卡和输入密码完成身份验证，系统验证通过后进入业务菜单。

主要成功场景：

1. 客户将银行卡插入 ATM。
2. ATM 系统读取卡号信息，并提示客户输入密码。
3. 客户输入密码。
4. ATM 系统将卡号、密码和设备上下文提交给认证模块。
5. 认证模块调用银行核心系统或账户数据完成校验。
6. 系统返回认证成功结果。
7. ATM 系统建立当前会话，记录必要的登录日志，并显示主菜单。

扩展与异常流程：

1. 若银行卡状态异常、卡片不可读或已被停用，则系统提示该卡不可用，并结束本次认证。
2. 若密码错误，则系统提示重新输入；连续多次输入错误时，系统终止本次会话并退卡。
3. 若客户在输入过程中长时间无操作，则系统自动取消认证并回到安全状态。
4. 若后台认证服务不可用，则系统提示暂时无法登录，不得泄露内部异常细节。

补充性要求与业务规则：

- 密码输入应采用遮蔽显示，错误提示不得暴露敏感认证信息。
- 只有建立有效会话后，客户才能访问余额查询和交易类功能。
- 认证失败后的系统状态必须可恢复，且不应残留脏会话。

分析说明：登录认证是后续所有业务的入口，对界面流程、会话管理、异常处理和外部接口依赖都有直接影响，因此属于初始阶段必须优先澄清的关键用例。

1.5.2 用例二：查询余额

- **用例名称：**查询余额
- **用例编号：**SUC002
- **主要参与者：**客户
- **目标：**在不改变账户状态的前提下，向客户展示当前登录账户的可用余额信息。
- **触发条件：**客户在主菜单中选择“查询余额”。
- **前置条件：**客户已完成登录认证，当前会话有效。
- **后置条件：**系统展示余额结果，并保留客户继续返回菜单或结束操作的选择。
- **用例简述：**客户通过 ATM 查询当前账户的可用余额。

主要成功场景：

1. 客户在主菜单中选择“查询余额”。
2. ATM 系统向账户服务发起余额查询请求。
3. 账户服务依据当前会话获取对应账户信息。
4. 系统读取账户余额数据并完成结果格式化。
5. ATM 系统将余额结果展示给客户。

6. 客户可选择返回主菜单或结束本次操作。

扩展与异常流程：

1. 若会话已失效，则系统提示重新登录。
2. 若账户信息不存在或账户状态异常，则系统提示查询失败，并回到安全菜单。
3. 若后台服务或网络异常，则系统提示稍后重试。

补充性要求与业务规则：

- 余额查询必须基于有效会话，且只允许查询当前登录账户。
- 界面展示应采用清晰的金额格式，并避免显示完整敏感账户标识。
- 作为基础只读业务，查询操作应保持较好响应速度和高可理解性。

分析说明：查询余额虽然业务复杂度较低，但它验证了菜单导航、会话控制、后台查询接口与结果展示链路是否连通，是后续扩展其他查询类和展示类功能的重要基础。

1.5.3 用例三：取款

- **用例名称：**取款
- **用例编号：**SUC003
- **主要参与者：**客户
- **目标：**在满足金额规则、账户余额和设备现金条件的前提下，完成扣款、吐钞与流水记录。
- **触发条件：**客户登录后选择“取款”功能。
- **前置条件：**客户已完成登录认证；账户状态正常；ATM 设备可用且现金充足。
- **后置条件：**成功时账户余额被扣减、系统生成交易记录，并可选择打印凭条；失败时不得产生未记录的异常账务状态。
- **用例简述：**客户在 ATM 上输入取款金额，系统校验通过后完成扣款和吐钞。

主要成功场景：

1. 客户进入主菜单并选择“取款”。
2. ATM 系统提示客户输入取款金额。
3. 客户输入金额并确认。
4. 系统校验金额格式及 ATM 业务金额规则。

5. 系统校验账户余额是否充足。
6. 系统校验 ATM 当前现金是否充足。
7. 系统提交扣款请求并获得成功结果。
8. ATM 执行吐钞。
9. 系统记录交易流水，并提示客户是否打印凭条。
10. 客户选择打印凭条或直接返回菜单。

扩展与异常流程：

1. 若金额不合法，则系统提示重新输入。
2. 若余额不足，则系统提示交易失败，不得扣款。
3. 若 ATM 现金不足，则系统提示暂时无法完成取款。
4. 若交易处理过程中后台异常、网络异常或设备异常，则系统应回滚或进入可对账的异常处理状态，并提示失败。
5. 若客户选择打印凭条，则系统在交易完成后触发打印；若打印失败，不影响已完成交易，但应给出提示。

补充性要求与业务规则：

- 取款金额必须满足 ATM 业务金额规则；结合实验要求示例，可初步约定最低 50 元、最高 5000 元，并以 50 元为单位。
- 若后续细化阶段引入手续费，则交易款项可按“取款金额 + 手续费”处理，但本阶段先保留为可扩展规则。
- 吐钞前必须保证扣款成功，交易完成后必须记录流水。
- 系统不得出现“已扣款、未吐钞、无记录”的不一致状态。

分析说明：取款用例同时涉及金额规则、账户余额、设备状态、交易一致性和日志审计，是 ATM 系统中最具架构意义与风险暴露价值的核心用例之一，应在初始阶段优先获得较清晰的业务认识。

1.5.4 用例四：转账

- 用例名称：转账
- 用例编号：SUC004

- **主要参与者：** 客户
- **目标：** 将当前账户中的指定金额安全转入目标账户，并记录完整交易流水。
- **触发条件：** 客户在主菜单中选择“转账”。
- **前置条件：** 客户已完成登录认证；当前账户状态正常；目标账户可被校验。
- **后置条件：** 成功时完成转出账户扣款、目标账户入账和交易流水记录；失败时保持账务一致。
- **用例简述：** 客户通过 ATM 向其他账户发起转账。

主要成功场景：

1. 客户登录后选择“转账”功能。
2. 系统提示输入目标账户和转账金额。
3. 客户输入目标账户和金额并确认。
4. 系统校验目标账户是否存在且可入账。
5. 系统校验当前账户余额是否充足。
6. 系统执行扣款与目标账户入账。
7. 系统记录交易流水，并向客户反馈转账成功结果。
8. 客户可选择打印凭条或返回主菜单。

扩展与异常流程：

1. 目标账户不存在、状态异常或不允许转入时，系统提示失败。
2. 当前账户余额不足时，系统提示失败且不得扣款。
3. 若客户输入的目标账户与当前账户相同，则系统拒绝提交。
4. 若后台处理异常，则系统提示转账失败，并保证转出与转入操作的一致性。

补充性要求与业务规则：

- 转账金额必须为合法正数，并满足银行或课程项目预设的限额规则。
- 扣款与入账应被视为同一业务事务，不能只完成其中一部分。
- 转账结果应生成唯一交易编号并进入交易流水，以支持后续查询和审计。

分析说明：转账用例比取款进一步放大了系统对外部账户信息、事务一致性和错误恢复的要求，是验证后端服务边界和业务原子性的关键场景，对后续设计类图、接口设计和异常处理策略都有直接影响。

1.6 本章小结与补充说明

本章围绕 ATM 系统的初始阶段完成了较完整的需求起步工作：一方面，从系统背景、课程定位和阶段目标出发，明确了本项目并非在初始阶段穷尽全部需求，而是建立共同设想、界定合理范围、识别关键风险并为后续迭代做准备；另一方面，结合实验要求和作业方案，识别了客户、银行核心系统、ATM 设备和管理员等主要参与者，明确了系统边界、功能范围以及 FURPS+ 视角下的关键补充性需求。

在此基础上，本文建立了 ATM 系统的总体用例视图，并按照“优先处理高价值、高风险、具有架构意义的代表性用例”的原则，对登录认证、查询余额、取款和转账四个核心用例进行了较详细的规约说明。这些内容将作为后续分析类图、活动图、顺序图和设计模型的输入。下一阶段将在此基础上继续细化显示模块与核心业务服务之间的职责分配和对象协作。

第二章 细化迭代 1

根据总体迭代计划，第二章对应细化迭代 1。本阶段不再停留在“列出需求和参与者”的初始层面，而是在第一章已经形成的系统背景、系统边界和关键用例基础上，进一步落实领域模型、系统顺序图、操作契约、包图、交互图和类图等 UML 制品。结合实验要求中“主要实现显示模块的分析设计与建模”的说明，本章把显示模块作为第一次迭代的主线：前端界面负责接收客户操作、展示业务结果、维护会话状态并调用后端接口；后端认证、账户和交易模块则为显示模块提供可调用、可验证、可继续扩展的服务边界。

需要说明的是，细化迭代 1 并不等同于完成全部 ATM 业务。按照 UP 的迭代思想，本阶段更适合先稳定高价值、高风险和高架构影响的核心链路，即登录认证、主菜单导航、账户信息查询和余额查询。取款、存款、转账等交易功能在本阶段已经完成接口路径、请求对象、响应对象、Controller 和 Service 骨架接入，但完整金额校验、事务处理、交易流水持久化、设备状态联动等内容仍保留到后续细化迭代中继续完善。

本章写法延续第一章“先说明阶段目标，再说明模型与代码依据，最后说明边界和验收”的表达方式。文档重点不是把后续迭代尚未完成的能力提前写成已实现功能，而是准确说明第一次迭代已经完成什么、为什么这样划分、以及这些制品如何支撑下一次迭代。

本次迭代后的工程基线如下：前端位于 frontend 目录，已包含 WelcomeView、LoginView、MenuView、BalanceView、PlaceholderView，以及 router、Pinia session store 和 api/http 封装；后端主模块为 atm-server-auth，包含 AuthController、AccountController、AuthServiceImpl、AccountServiceImpl、TokenManager、Mapper、实体与 DTO；交易处理模块位于 transaction 目录，已经通过 Maven 编译配置接入主后端，并提供 TransactionController、TransactionService、TransactionServiceImpl、交易 DTO 与 Transaction 实体骨架。

2.1 本次迭代目标与实现范围

细化迭代 1 的目标可以概括为“建立可运行的最小业务闭环，并把后续交易功能需要依赖的模型和接口边界先固定下来”。从课程制品角度看，本章必须覆盖 Domain Models、System Sequence Diagrams、Operation Contracts、UML Package Diagrams、UML Interaction Diagrams 和 UML Class Diagrams；当前版本已经可以通过 dev 内存数据库完成登录、会话建立、账户信息查询和余额查询；主菜单已预留取款、存款、转账、修改密码和凭条入口；交易接口已挂载到 `/api/atm/transactions` 路径下，但服务实现按第一次迭代计划返回 501，表示完整业务逻辑将在后续迭代实现。从项目实施角度看，本阶段已经完成登录、退出、账户信息查询、余额查询以及交易模块骨架接入。二者之间的对应关系如下表所示。

迭代计划要求	本章对应制品	项目实际依据
Domain Models（领域模型）	2.3 领域模型图与领域对象说明	Customer、BankCard、Account、Session、Transaction、Receipt、ATMDevice 等领域概念；当前代码已落地账户、银行卡、会话和交易骨架相关结构。
System Sequence Diagrams（系统顺序图）	2.4 登录、余额查询、交易骨架系统顺序图	WelcomeView、LoginView、MenuView、BalanceView、PlaceholderView 与后端 Controller/Service 的调用链路。
Operation Contracts（操作契约）	2.5 登录、退出、账户查询、余额查询、交易骨架操作契约	AuthController、AccountController、TransactionController 对外暴露的系统操作。
UML Package Diagrams（包图）	2.6 第一次迭代包图	frontend、atm-server-auth、transaction 三部分目录及其依赖关系。
UML Interaction Diagrams（交互图）	2.7 登录与余额查询协作图、交易骨架协作图	Controller-Service-Mapper-DTO-Entity-TokenManager 之间的对象协作。
UML Class Diagrams（类图）	2.8 后端核心类图	认证、账户、交易模块中最核心的 Java 类、接口、DTO 与公共响应结构。
实现与验收	2.9 本次迭代实现与验收说明	前端构建、后端测试、登录余额查询链路以及交易接口 501 边界验证。

2.2 显示模块的第一次迭代职责

实验要求明确强调“主要实现显示模块”的分析设计与建模，因此本章不把显示模块简单理解为若干页面文件，而是把它看作 ATM 系统中最直接面向客户的边界层。显示模块承担四类职责：第一，向客户展示当前系统状态和可选业务；第二，接收客户输入并进行基础的前端校验；第三，维护登录成功后的 `sessionId` 和页面跳转关系；第四，通过统一 API 封装调用后端认证、账户与交易服务。

显示模块组成	当前职责	第一次迭代状态
WelcomeView	展示系统入口，引导客户进入登录流程。	已实现，可作为演示起点。
LoginView	接收卡号和密码，调用登录接口，登录成功后保存 <code>sessionId</code> 并跳转主菜单。	已实现，是第一次迭代最重要的入口界面。
MenuView	展示查询余额、取款、存款、转账、修改密码、凭条等功能入口。	已实现核心导航，交易类入口暂指向占位或骨架流程。
BalanceView	读取 <code>sessionId</code> ，调用余额查询接口并展示结果。	已实现，可验证前后端、会话和账户服务闭环。
PlaceholderView	承接尚未完整实现的取款、存款、转账、修改密码和凭条功能。	已实现占位提示，避免把未完成业务误表现为已完成能力。
router、Pinia store、api/http	负责页面导航、会话状态保存、统一 HTTP 请求和错误提示。	已实现基础封装，是显示模块与后端模块解耦的关键。

在第一次迭代中，显示模块最重要的价值不在于页面数量，而在于它把客户视角下的“登录 - 主菜单 - 查询余额 - 返回/退出”流程跑通，并为交易类功能保留一致的入口和接口调用方式。后续细化迭代只需要在现有入口下继续填充交易规则和结果展示，而不需要重新调整整体导航结构。

因此，本章后续 UML 图均会把显示模块作为边界对象处理：在系统顺序图中它负责发起系统事件；在交互图中它负责调用 API 封装并保存会话；在包图中它属于 `frontend` 包；在类图中则主要体现为前端与 `Controller` 之间的接口边界，而不是把每个 `Vue` 组件都建成复杂的业务类。

2.3 领域模型

领域模型用于描述 ATM 系统中较稳定的业务概念及其关系，而不是直接照搬数据库表、接口参数或界面控件。结合第一章的用例规约、实验要求中的 ATM 业务范围以及当前项目代码，第一次迭代的领域模型以客户、银行卡、账户、会话和交易为核心。其中 Customer、BankCard、Account 和 Session 支撑当前已经跑通的登录和查询余额链路；Transaction、Receipt 和 ATMDDevice 在本阶段主要承担概念建模与接口扩展作用，为后续真实交易处理、凭条打印和设备状态反馈预留位置。

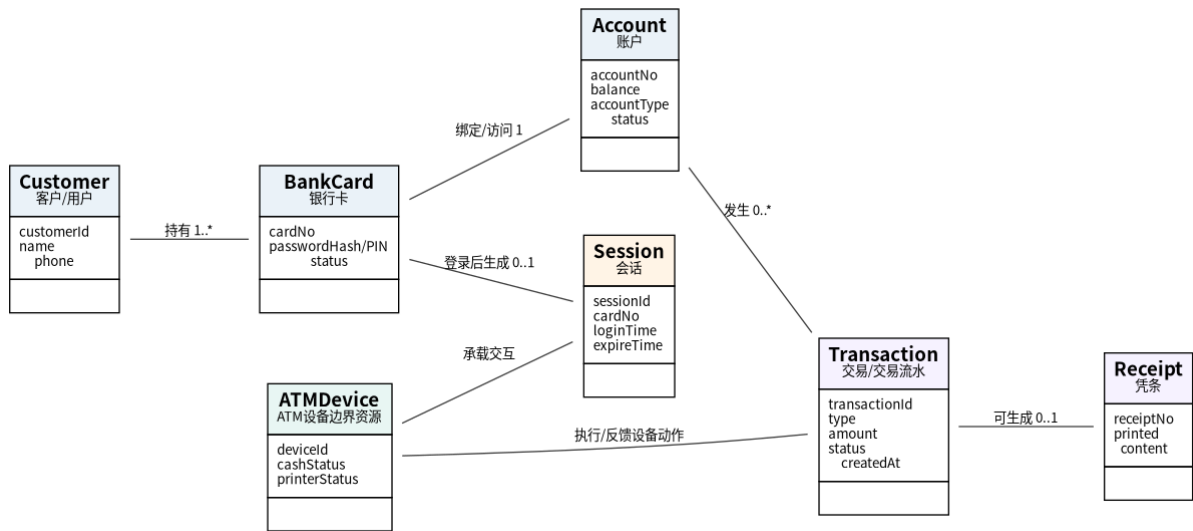


图 2-1 ATM 系统第一次迭代领域模型

从模型关系看，客户可以持有一张或多张银行卡，银行卡绑定可访问的账户；客户通过银行卡登录成功后，系统生成会话；账户是余额查询和交易处理的核心主体；交易可进一步关联凭条和设备动作。当前代码已经支撑登录会话和账户余额查询，而交易与凭条更多体现为下一阶段要补全的领域对象。

领域概念	英文标识	第一次迭代中的作用
客户	Customer / User	表示 ATM 服务的直接使用者，是登录、查询和交易请求的业务主体。
银行卡	BankCard	承载卡号、密码校验和卡片状态，是登录认证链路的关键对象。
账户	Account	保存账户号、余额、类型和状态，是余额查询与后续交易处理的核心对象。
会话	Session	由 TokenManager 维护，以 sessionId 形式传递，是前端访问后续接口的身份凭证。
交易	Transaction	表示取款、存款、转账等会改变账户状态或形成流水的业务事件；第一次迭代为骨架。
凭条	Receipt	表示业务完成后的打印结果或查询凭证；第一次迭代只保留为后续扩展概念。

ATM 设备	ATMDevice	表示读卡、吐钞、打印和设备状态反馈等边界资源；第一次迭代暂不实现真实硬件控制。
--------	-----------	---

与第一章相比，本节的领域模型更加偏向“后续设计可用”的结构。第一章关注系统边界和主要用例，本节则开始明确哪些概念会在后端实体、DTO、会话对象、接口参数和数据库结构中中长期存在。这样处理可以避免后续章节在补充交易流水和凭条功能时重新命名或重新划分对象。

2.4 系统顺序图

系统顺序图从系统边界角度描述外部参与者向系统发出的事件顺序。考虑到本项目采用前后端分离结构，本节将客户、显示模块、前端 API 封装、后端 Controller、Service、Mapper/DB 和 TokenManager 作为主要交互对象。登录认证和查询余额是当前可运行能力，交易类操作在本迭代中只体现为接口骨架与 501 响应。

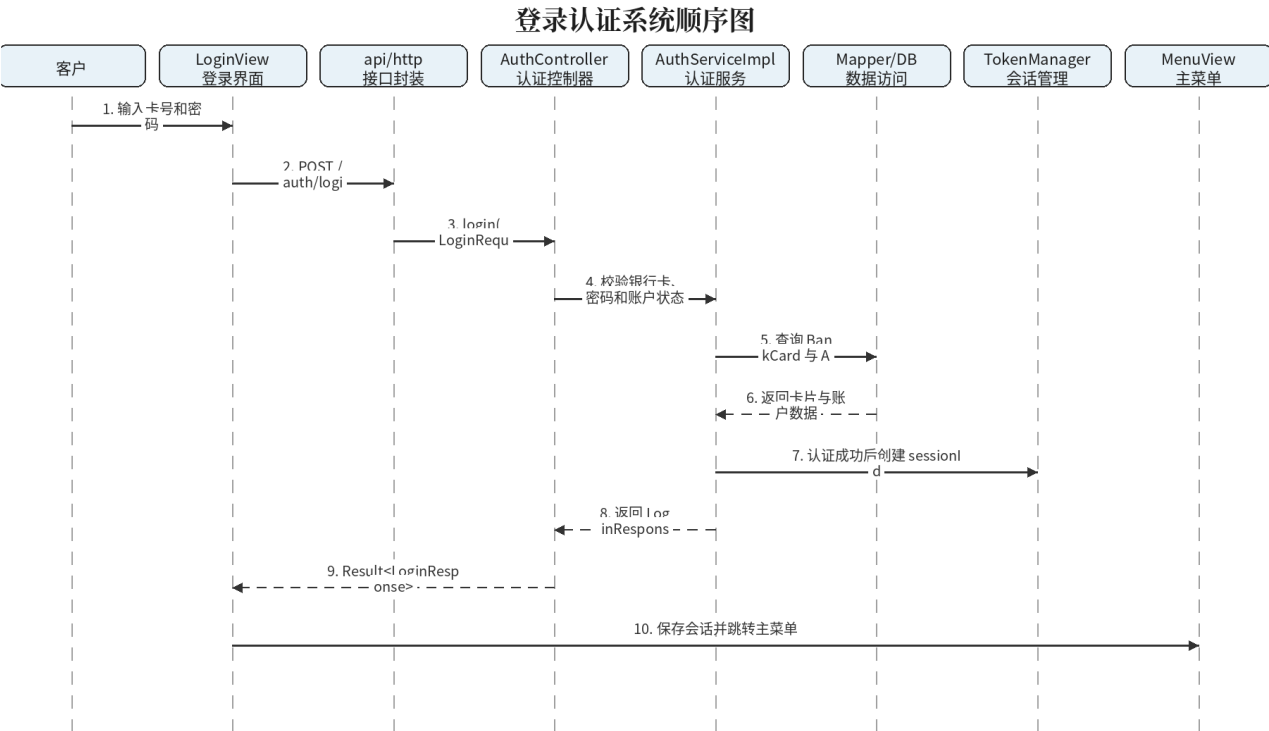


图 2-2 登录认证系统顺序图

登录认证顺序图反映了第一次迭代的入口闭环。客户在 LoginView 输入卡号和密码后，前端通过 api/http 封装发送请求，后端 AuthController 将请求转交给 AuthServiceImpl，服务层读取

银行卡和账户数据，验证通过后由 TokenManager 生成 sessionId。前端保存 sessionId 后跳转到主菜单。该链路同时验证了显示模块、认证服务、数据访问和会话管理之间的基本协作。

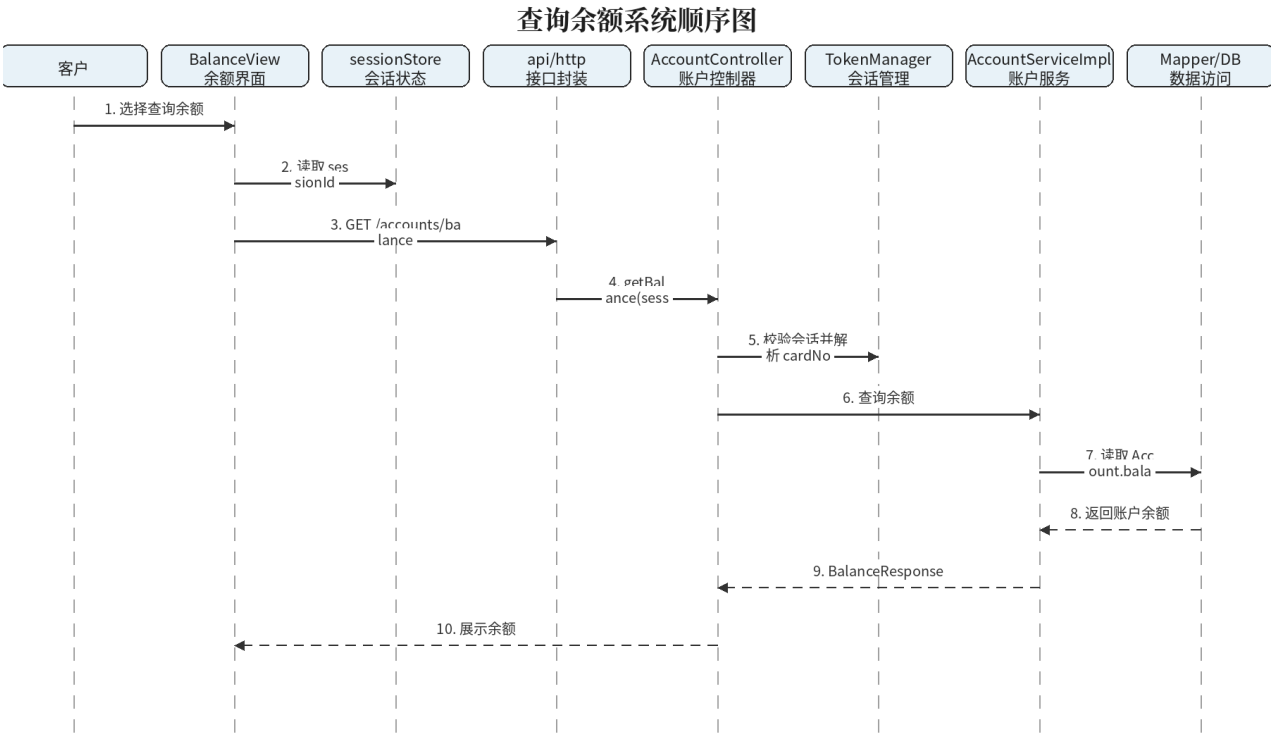


图 2-3 查询余额系统顺序图

余额查询顺序图体现了登录之后的典型只读业务。BalanceView 不直接持有卡号或账户号，而是从 Pinia session store 中读取 sessionId，再通过 AccountController 和 TokenManager 解析当前会话对应的卡号，最后由账户服务读取余额并返回 BalanceResponse。这样的设计可以减少前端传递敏感字段，也为后续取款、存款和转账接口统一使用 sessionId 奠定基础。

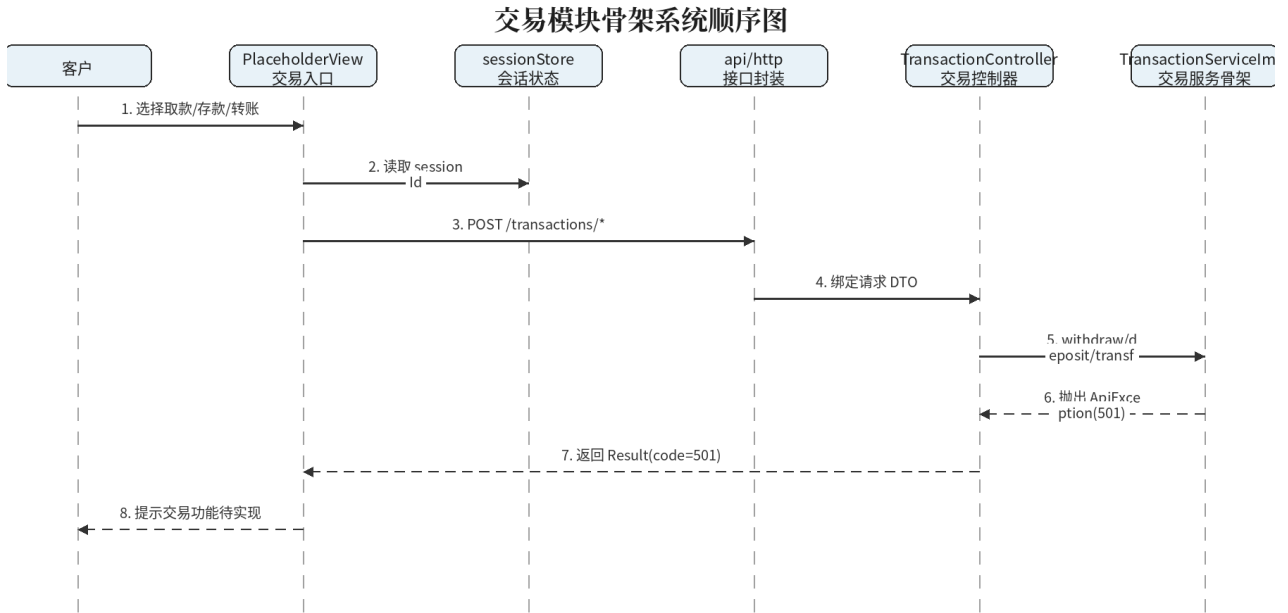


图 2-4 交易模块骨架系统顺序图

交易模块骨架顺序图特意保留了 `TransactionServiceImpl` 返回 501 Not Implemented 的事实。这不是运行错误，而是第一次迭代的边界说明：交易接口已经被 `Spring MVC` 挂载，前端也可以按统一口径提交 `sessionId` 和业务参数，但真实交易规则、事务控制和流水写入尚未在本阶段完成。这样写可以保证文档与代码一致，避免验收时出现“文档写了完整交易，代码只完成骨架”的不一致问题。

2.5 操作契约

操作契约描述系统操作执行前后的状态变化、输入输出、异常约束和实现边界。与第一章的用例规约相比，操作契约更接近设计层，它回答的是“系统收到某个操作后，哪些对象状态应当改变，哪些状态不应当改变”。本节按当前代码实际提供的系统操作进行整理。

系统操作	login(cardNo, password)
关联用例	登录认证
职责说明	客户提交银行卡号和密码，系统完成身份校验，并在成功时建立有效 ATM 会话。
前置条件	客户位于登录界面；cardNo 和 password 已输入；后端认证服务可用；演示数据中存在对应银行卡和账户记录。
后置条件	认证成功时生成 sessionId，前端保存会话并进入主菜单；认证失败时不建立会话，不跳转主菜单。
异常/替代结果	密码错误、卡片不存在、账户异常或认证服务不可用时返回失败结果；错误信息应可理解，但不暴露敏感认证细节。
当前代码依据	LoginView -> atm.js/http.js -> AuthController.login -> AuthServiceImpl.login -> BankCardMapper/AccountMapper -> TokenManager。
第一次迭代边界	已完整实现并可演示，是第一次迭代验收的核心链路。
系统操作	logout(sessionId)
关联用例	退卡/结束会话
职责说明	客户结束本次 ATM 操作，系统释放当前会话并返回安全状态。
前置条件	客户已登录；前端保存 sessionId；后端 TokenManager 中存在或曾存在该会话。
后置条件	服务端移除对应 sessionId；前端清除本地会话并返回登录或欢迎界面。
异常/替代结果	若 sessionId 已失效，系统仍应保证前端进入未登录状态，不继续允许访问账户类接口。
当前代码依据	MenuView/前端退出口 -> AuthController.logout -> AuthServiceImpl.logout -> TokenManager.remove。
第一次迭代边界	已实现基础退出逻辑；真实 ATM 退卡动作属于后续设备状态模拟范围。
系统操作	getProfile(sessionId)
关联用例	查询账户信息
职责说明	系统根据当前会话返回必要的账户概要信息，用于主菜单或个人信息展示。
前置条件	客户已登录；sessionId 有效；账户记录存在。
后置条件	返回当前会话对应账户的基本信息；不修改账户余额和交易流水。
异常/替代结果	sessionId 无效或账户不存在时返回未授权或查询失败结果。
当前代码依据	AccountController.getProfile -> TokenManager -> AccountServiceImpl.getProfile -> AccountMapper。
第一次迭代边界	已实现基础查询；展示字段以演示系统需要为准，暂不包含复杂客户画像和风险信息。
系统操作	getBalance(sessionId)
关联用例	查询余额
职责说明	系统在不改变账户状态的前提下，向显示模块返回当前账户余额。
前置条件	客户已登录；sessionId 有效；账户状态允许查询。

后置条件	返回 BalanceResponse，前端展示余额；账户余额、交易流水和会话状态不发生业务性修改。
异常/替代结果	sessionId 无效时返回 401；账户异常或后台错误时返回失败提示。
当前代码依据	BalanceView -> AccountController.getBalance -> TokenManager -> AccountServiceImpl.getBalance -> AccountMapper。
第一次迭代边界	已完整实现并可演示，是第一次迭代除登录外最重要的验收链路。
系统操作	withdraw(sessionId, amount, printReceipt)
关联用例	取款
职责说明	业务目标是在后续迭代中完成金额校验、余额校验、扣款、吐钞、流水记录和可选凭条打印。
前置条件	客户已登录；请求包含 sessionId、amount 和 printReceipt；交易接口已挂载。
后置条件	第一次迭代中不修改账户余额，不生成真实交易流水；返回 501 表示功能待实现。
异常/替代结果	若请求到达交易服务，统一返回 Not Implemented；后续迭代再补充金额不合法、余额不足、设备现金不足等替代流程。
当前代码依据	TransactionController.withdraw -> TransactionServiceImpl.withdraw -> ApiException(501)。
第一次迭代边界	仅完成接口、DTO、Controller、Service 骨架，不声明真实取款能力已完成。
系统操作	deposit(sessionId, amount, printReceipt)
关联用例	存款
职责说明	业务目标是在后续迭代中完成金额校验、入账、流水记录和可选凭条打印。
前置条件	客户已登录；请求包含 sessionId、amount 和 printReceipt；交易接口已挂载。
后置条件	第一次迭代中不修改账户余额，不生成真实交易流水；返回 501 表示功能待实现。
当前代码依据	TransactionController.deposit -> TransactionServiceImpl.deposit -> ApiException(501)。
第一次迭代边界	仅完成接口、DTO、Controller、Service 骨架，不声明真实存款能力已完成。
系统操作	transfer(sessionId, targetAccountNo, amount, printReceipt)
关联用例	转账
职责说明	业务目标是在后续迭代中完成目标账户校验、余额校验、转出与转入事务处理、流水记录和凭条处理。
前置条件	客户已登录；请求包含 sessionId、目标账户、金额和凭条选项；交易接口已挂载。
后置条件	第一次迭代中不执行真实转账，不修改转出或转入账户余额；返回 501。
异常/替代结果	后续迭代需要补充目标账户不存在、目标账户不可入账、余额不足、事务失败回滚等异常流程。
当前代码依据	TransactionController.transfer -> TransactionServiceImpl.transfer -> ApiException(501)。
第一次迭代边界	仅完成接口骨架和统一响应，不声明转账业务已经实现。

上述契约体现了本次迭代的两个重点：一是登录、账户信息和余额查询已经具备明确的前置条件、后置条件和异常处理；二是交易类操作虽然已经拥有稳定的系统操作名称和接口路径，

但其业务后置条件在第一次迭代中被明确限制为“不修改账户状态、不生成真实流水”。这种写法有助于后续迭代继续扩展，同时不会造成文档与实现之间的矛盾。

2.6 UML 包图

包图用于说明系统的高层组织结构和依赖方向。本项目第一次迭代的包结构体现了前后端分离和小组分工边界：**frontend** 负责显示模块、路由、状态管理和 HTTP 调用；**atm-server-auth** 是主 Spring Boot 后端，负责认证、账户、公共响应和内存会话管理；**transaction** 是交易模块骨架，通过 Maven 构建配置接入主后端应用。

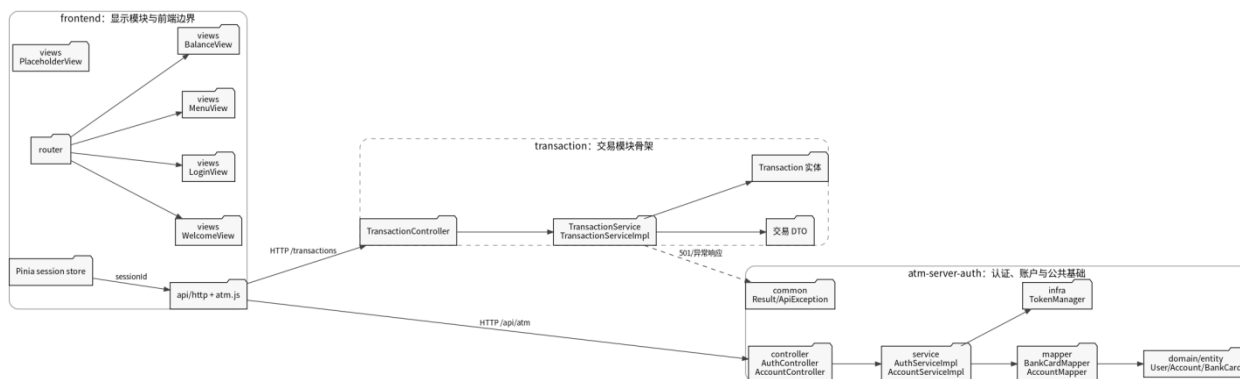


图 2-5 第一次迭代 UML 包图

从包图可以看出，**frontend** 不直接访问数据库，也不直接处理银行卡、账户和交易对象，而是通过统一 API 封装调用后端接口。**atm-server-auth** 承担当前可运行能力的主体责任；**transaction** 保持相对独立，既便于小组成员并行开发，也便于后续把真实交易逻辑逐步补入 **TransactionServiceImpl**。包之间依赖方向总体保持从显示层到控制层、服务层、数据访问层的单向调用，符合低耦合和职责分离的设计要求。

2.7 UML 交互图

交互图进一步从对象协作角度解释当前代码如何完成业务请求。与系统顺序图强调“外部事件顺序”不同，交互图更关注系统内部对象之间的职责分配。登录与余额查询已经形成 Controller-Service-Mapper-Entity-TokenManager 的闭环；交易模块则形成 Controller-Service-DTO-统一异常响应的骨架协作。

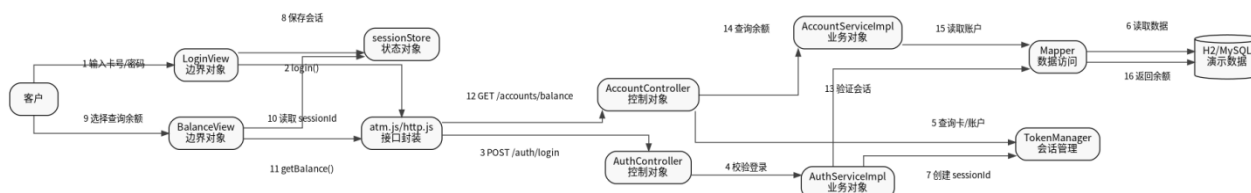


图 2-6 登录与余额查询 UML 交互图

图 2-6 中，LoginView 和 BalanceView 属于边界对象，负责与客户直接交互；AuthController 和 AccountController 属于控制对象，负责接收 HTTP 请求并协调业务服务；AuthServiceImpl 和 AccountServiceImpl 属于业务对象，负责处理认证与账户查询规则；Mapper 和数据库承担数据访问职责；TokenManager 负责会话创建、解析和移除。该协作结构与第一章中“显示模块与认证、账户服务之间的交互关系”相对应。

交易模块骨架 UML 交互图

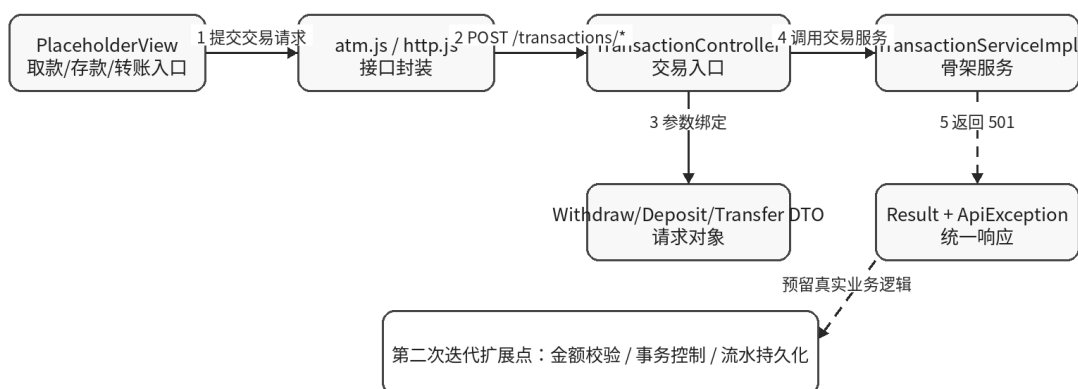


图 2-7 交易模块骨架 UML 交互图

图 2-7 表明交易模块在第一次迭代中已经具备可调用入口，但服务层仍是占位实现。后续细化迭代 2 的主要任务是在 TransactionServiceImpl 内补充金额规则、余额校验、交易编号生成、交易流水持久化和事务一致性处理；显示模块则在现有 PlaceholderView 或交易页面基础上继续完善输入界面和结果展示界面。

2.8 UML 类图

类图选取当前后端最核心的类和接口，重点展示认证、账户与交易模块之间的职责边界。**Controller** 负责 HTTP 请求入口，**Service/ServiceImpl** 负责业务规则或骨架占位，**Mapper** 负责数据访问，**TokenManager** 维护会话，**DTO** 与 **Result** 负责接口数据结构。由于第一次迭代强调“与代码一致”，本节不把尚未实现的完整交易事务类、设备驱动类或凭条打印类提前写成可用设计类。

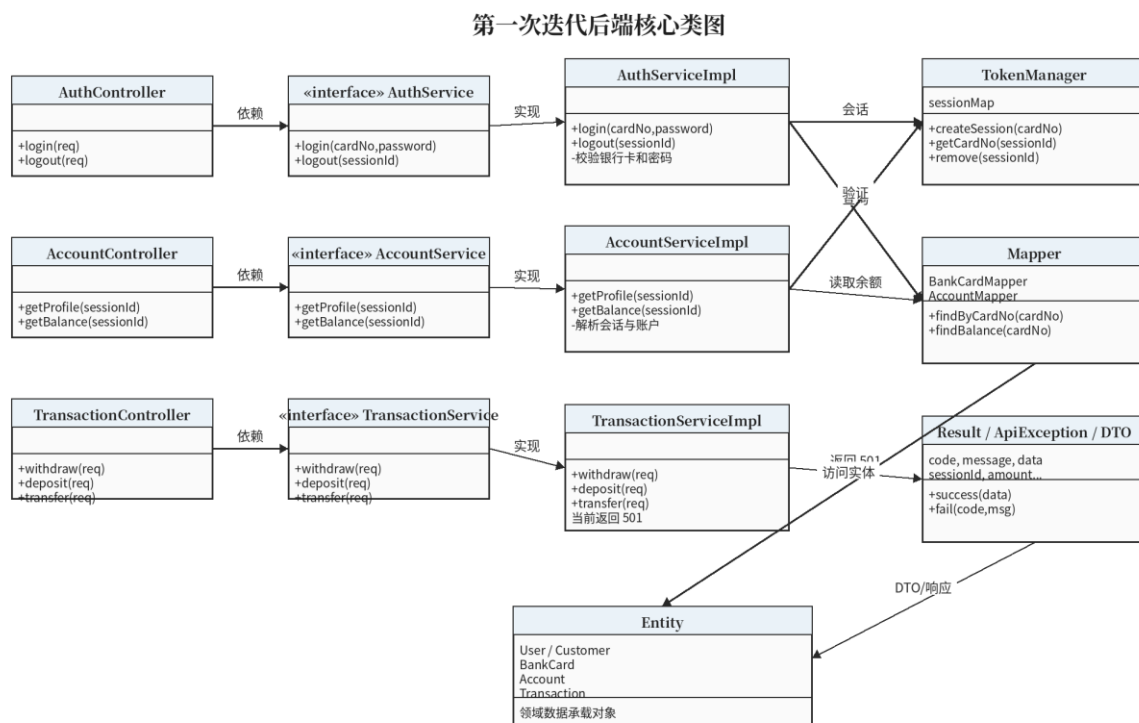


图 2-8 第一次迭代后端核心类图

从类图可以看出，认证与账户模块已经具备从 **Controller** 到 **ServiceImpl** 再到 **Mapper** 的可执行链路；交易模块目前保持接口和 **DTO** 先行的骨架形态。**Result**、**ApiException** 等公共结构为三个模块提供统一响应口径，**TokenManager** 则使前端只需要持有 `sessionId`，而不必在后续请求中反复传递 `cardNo`。这样的类结构有利于后续在不破坏接口口径的前提下继续实现取款、存款和转账。

2.9 本次迭代实现与验收说明

本次迭代的可运行部分包括：前端通过 Vite 启动后能够访问欢迎页、登录页、主菜单页和余额查询页；后端通过 dev 配置启动后能够使用 H2 内存数据库初始化演示客户、账户和银行卡；认证接口、退出接口、账户信息接口、余额接口和交易骨架接口均能被 Spring MVC 挂载。交易模块返回 501 并非运行错误，而是第一次迭代交付边界的一部分。

验收项	操作方式	预期结果
前端启动与构建	进入 frontend 目录执行 npm run build，或本地启动后访问 http://localhost:5173 。	欢迎页、登录页、主菜单页和余额查询页可正常进入，页面路由无明显异常。
登录认证链路	使用演示卡号 6222020000000001 和密码 123456 登录。	后端返回登录成功结果，前端保存 sessionId 并进入主菜单。
余额查询链路	登录成功后点击“查询余额”。	前端携带 sessionId 调用余额接口，页面展示后端返回的余额数据。
交易骨架接口	访问 POST /api/atm/transactions/withdraw、deposit 或 transfer。	接口可被路由到交易 Controller，服务层返回 501，表示交易完整逻辑将在后续迭代实现。
后端测试	在 atm-server-auth 目录执行 .mvnw.cmd test 或等价 Maven 测试命令。	认证、账户和交易骨架 Controller 测试通过，主后端能够编译交易模块代码。

本阶段验收时应重点检查“文档、模型和代码是否一致”。如果交易接口返回 501，不应判定为失败；只有在接口未挂载、项目无法编译、登录无法获取 sessionId、余额查询无法基于 sessionId 返回结果时，才说明第一次迭代的核心闭环存在问题。

后续迭代建议在当前基线之上补充三类内容：其一，完善取款、存款、转账的金额规则和事务处理；其二，增加 transaction_record 持久化、交易编号和交易详情查询；其三，补充设备状态、凭条打印和异常回滚说明，使第二章中的骨架模型能够自然升级为完整业务模型。

2.10 本章小结

本章按照细化迭代 1 的要求，在第一章初始阶段文档基础上完成了更具体的分析与设计制品：领域模型说明了 ATM 系统中的核心业务概念；系统顺序图说明了客户、显示模块和后端服务之间的事件顺序；操作契约明确了登录、退出、账户查询、余额查询和交易骨架操作的前置条件、后置条件与实现边界；包图、交互图和类图进一步说明了当前工程的模块结构、对象协作和核心类职责。

从实现角度看，第一次迭代已经形成“登录认证 - 主菜单 - 查询余额”的可运行闭环，并且把交易模块以骨架形式接入主后端。这一结果符合 UP 细化阶段先处理核心风险、逐步演化模型的原则，也符合实验要求中按阶段提交主要制品、后续文档在前一版基础上升级完善的要求。下一章可在本章模型基础上继续推进细化迭代 2，重点补全取款、存款、转账、交易流水、凭条和异常处理等真实业务能力。

第三章 细化迭代 2

根据总体迭代计划，第三章对应“细化迭代 2”，提交内容被明确标注为 Applying GoF Design Patterns，提交时间为第 10 周周五晚。因此，本章不再重复第二章已经完成的领域模型、系统顺序图和包图等基础制品，而是在第一次迭代形成的代码基线之上，重点说明核心交易能力如何通过可维护的对象协作和设计模式思想落地。结合第二次迭代记录，本轮实现范围集中在取款、存款、转账、修改密码、账户完整信息、交易前校验和交易详情查询，并保持前端调用、后端 DTO、OpenAPI 和接口测试示例使用同一套公开契约。

本章写法延续前两章“先说明阶段目标，再说明模型与代码依据，最后说明边界、验收和后续演进”的风格。不同之处在于，第三章的中心不是枚举 UML 图，而是围绕 GoF 设计模式应用解释第二次迭代为什么这样组织交易模块、如何保证账务一致性、如何兼容旧接口并支撑后续凭条、流水和设备状态扩展。

3.1 本章定位与迭代依据

总体迭代计划中，第二章“细化迭代 1”要求提交 Domain Models、System Sequence Diagrams、Operation Contracts、UML Package Diagrams、UML Interaction Diagrams 和 UML Class Diagrams；而第三章“细化迭代 2”只列出 Applying GoF Design Patterns。这说明本阶段的核心任务不是重新绘制所有上一轮模型，而是在已有模型基础上进入设计决策层：识别哪些对象应该承担稳定职责，哪些变化点需要被封装，哪些跨模块调用需要通过统一接口协调，哪些兼容问题应由适配层处理。

从项目实现角度看，第二次迭代已经把第一次迭代中的交易接口骨架推进为真实业务闭环。前端新增取款、存款、转账和修改密码页面；后端补充完整账户信息、交易前校验、修改密码、取款、存款、转账和交易详情接口；交易服务通过事务控制完成账户余额更新与 transaction_record 交易流水写入；公开字段统一为 sessionId、transactionId 和 targetAccountNo。这些内容共同构成本章分析 GoF 模式应用的事实基础。

总体计划要求	第三章响应方式	项目实际依据
Applying GoF Design Patterns	分析 Facade、Proxy、Adapter、Template Method 思想、Factory/Creator 思想及容器单例管理	TransactionServiceImpl、SessionValidator、Spring @Transactional、MyBatis Mapper、Result、TransactionUtils
承接细化迭代 1	沿用第二章领域对象、系统操作、包结构和类图基线，不改变既有模块边界	frontend、atm-server-auth、transaction 三目录继续保持分工结构
服务第二次迭代实现	把取款、存款、转账、修改密码和交易详情写成可验证业务闭环	第二次迭代记录、api-test.http、openapi-atm.yaml、交易服务集成测试
为细化迭代 3 留出扩展	明确流水列表、凭条、设备状态、状态图和最终文档仍属后续阶段	三次迭代任务安排和第二次迭代记录中的未纳入范围

3.2 第二次迭代功能闭环

第二次迭代的直接目标是补全 ATM 系统的核心交易能力，使系统从“登录认证 - 主菜单 - 查询余额”的最小闭环，升级为“登录认证 - 账户校验 - 交易执行 - 余额更新 - 流水查询”的主要业务闭环。其中，取款、存款和转账会改变账户状态，因此比第一轮查询类功能更强调事务一致性、异常恢复、业务规则集中管理和接口契约统一。

模块	第二次迭代完成内容	设计关注点
前端显示模块	新增 WithdrawView、DepositView、TransferView、ChangePasswordView；主菜单入口改为真实业务页面；atm.js 统一封装交易和账户接口	边界对象只负责输入、提示、会话状态和接口调用，不承担账务规则
认证与账户模块	新增修改密码、完整账户信息、交易前校验；SessionValidator 统一校验 sessionId；账户接口主路径统一为 /api/atm/accounts/*	把身份解析和账户上下文获取作为交易前置能力，为交易服务提供稳定输入
交易处理模块	实现 withdraw、deposit、transfer、getTransactionById；写入 transaction_record；按单次/单日限额、余额、自转账、目标账户存在性进行校验	通过服务门面和事务代理隐藏复杂规则，保证接口简单且账务状态一致
文档与测试材料	OpenAPI、README、FUNCTIONS、api-test.http、第二次迭代记录同步更新；集成测试覆盖余额更新、异常拒绝和交易详情	让模型、接口、代码和验收材料保持同一事实口径

本轮特别需要说明的是，第二次迭代并未把交易流水列表、凭条展示、打印机状态、ATM 设备吐钞能力和完整设备异常处理提前写成已实现功能。这些内容虽然在第一章用例中已经出现，但按照三次迭代任务安排，它们应保留到细化迭代 3 或最终集成阶段。这样的边界划分既避免文档过度承诺，也符合 UP 迭代式逐步细化的思想。

3.3 GoF 模式应用总览

GoF 设计模式的价值不在于为每个类强行贴标签，而在于面对变化点时提供稳定的职责分配方式。在 ATM 第二次迭代中，主要变化点集中在交易类型差异、会话字段统一、数据库访问代理、事务边界、异常响应格式和后续凭条/设备扩展等方面。图 3-1 从前端、控制层、服务层、事务代理、数据访问层和后续扩展点几个角度，概括了本轮可以识别和应用的模式。

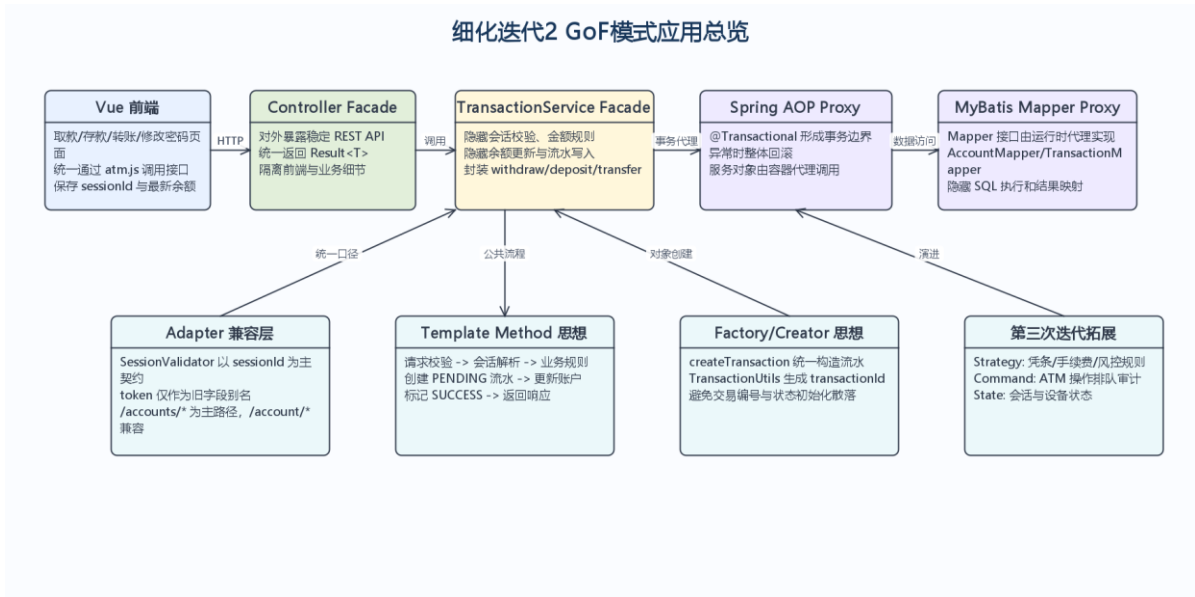


图 3-1 细化迭代 2 GoF 模式应用总览

模式或模式思想	本项目中的落点	解决的问题
Facade（外观模式）	TransactionController 和 TransactionService 对外暴露 withdraw、deposit、transfer、getTransactionById 等简单接口	隐藏会话校验、金额规则、Mapper 调用、流水写入和统一响应细节，降低前端与业务实现耦合
Proxy（代理模式）	Spring @Transactional 通过 AOP 代理形成事务边界；MyBatis Mapper 接口由框架生成运行时代理	把事务提交/回滚、SQL 执行和结果映射从业务代码中剥离出来
Adapter（适配器模式）	SessionValidator 将新 sessionId 和旧 token 兼容口径适配为业务服务需要的 cardNo	在不扩大业务服务复杂度的前提下完成字段统一和旧调用兼容
Template Method 思想	取款、存款、转账共享“请求校验 - 会话解析 - 账户上下文 - 规则校验 - 流水 - 余额更新 - 响应”骨架	让三类交易既保持公共处理流程，又允许差异化金额规则和目标账户规则
Factory/Creator 思想	createTransaction 和 TransactionUtils.generateTransactionId 统一构造交易流水和交易编号	避免交易对象初始化逻辑散落在多个分支中，便于后续扩展凭条或流水列表
容器管理单例	Controller、Service、Mapper、Validator 均由 Spring 管理，默认以单例 Bean 复用	避免手写 Singleton 带来的测试困难，同时保证核心服务对象生命周期稳定

其中，Proxy、Adapter 和 Facade 在当前代码中已有比较直接的落点；Template Method 和 Factory/Creator 更准确地说是“模式思想”在服务实现中的体现，当前通过私有公共方法和统一构造入口完成。后续如果交易种类继续增加，可进一步把这些思想重构为更严格的抽象模板、策略接口或命令对象。

3.4 Facade 与 Template Method 思想：交易服务设计

第二次迭代后，交易模块不再只是 Controller 和 DTO 的占位接口，而是承担了真实业务规则。为了不让前端页面或 Controller 直接感知银行卡、账户、交易流水和异常处理细节，本轮将 TransactionService 作为交易业务的服务门面：前端只需要提交 sessionId、amount、printReceipt 和必要的 targetAccountNo；Controller 只负责接收请求并包装 Result；TransactionServiceImpl 则在内部完成会话校验、账户上下文加载、业务规则判断、交易对象创建、余额更新、流水写入和响应组装。

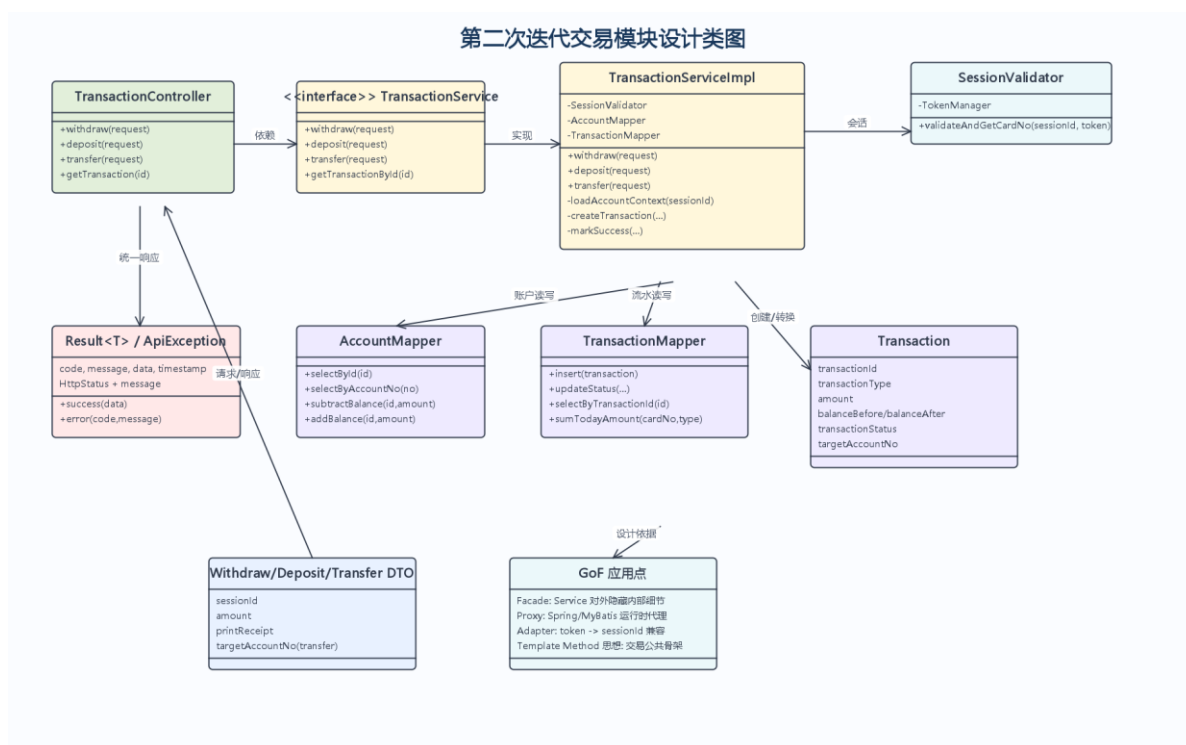


图 3-2 第二次迭代交易模块设计类图

图 3-2 展示了第二次迭代交易模块的主要设计类。TransactionController 依赖 TransactionService 接口，而不是直接依赖具体实现；TransactionServiceImpl 作为业务实现类，组合 SessionValidator、AccountMapper、BankCardMapper 和 TransactionMapper。这一结构具有明显的 Facade 效果：对外接口保持简洁稳定，对内则可以容纳金额规则、日累计限额、余额变更、交易状态和异常分支等复杂逻辑。

在实现层面，取款、存款和转账具有相同的交易骨架：先校验请求体和金额，再通过 sessionId 获取当前银行卡和账户，再执行交易类型相关规则，随后创建 PENDING 状态交易流水，更新账户余额，最后将交易标记为 SUCCESS 并返回 transactionId 与最新余额。差异点则集中在取款金额必须为 100 的整数倍、取款单次和单日限额、存款单次上限、转账目标账户存在性、自转账禁止、转账单次和单日限额等规则上。

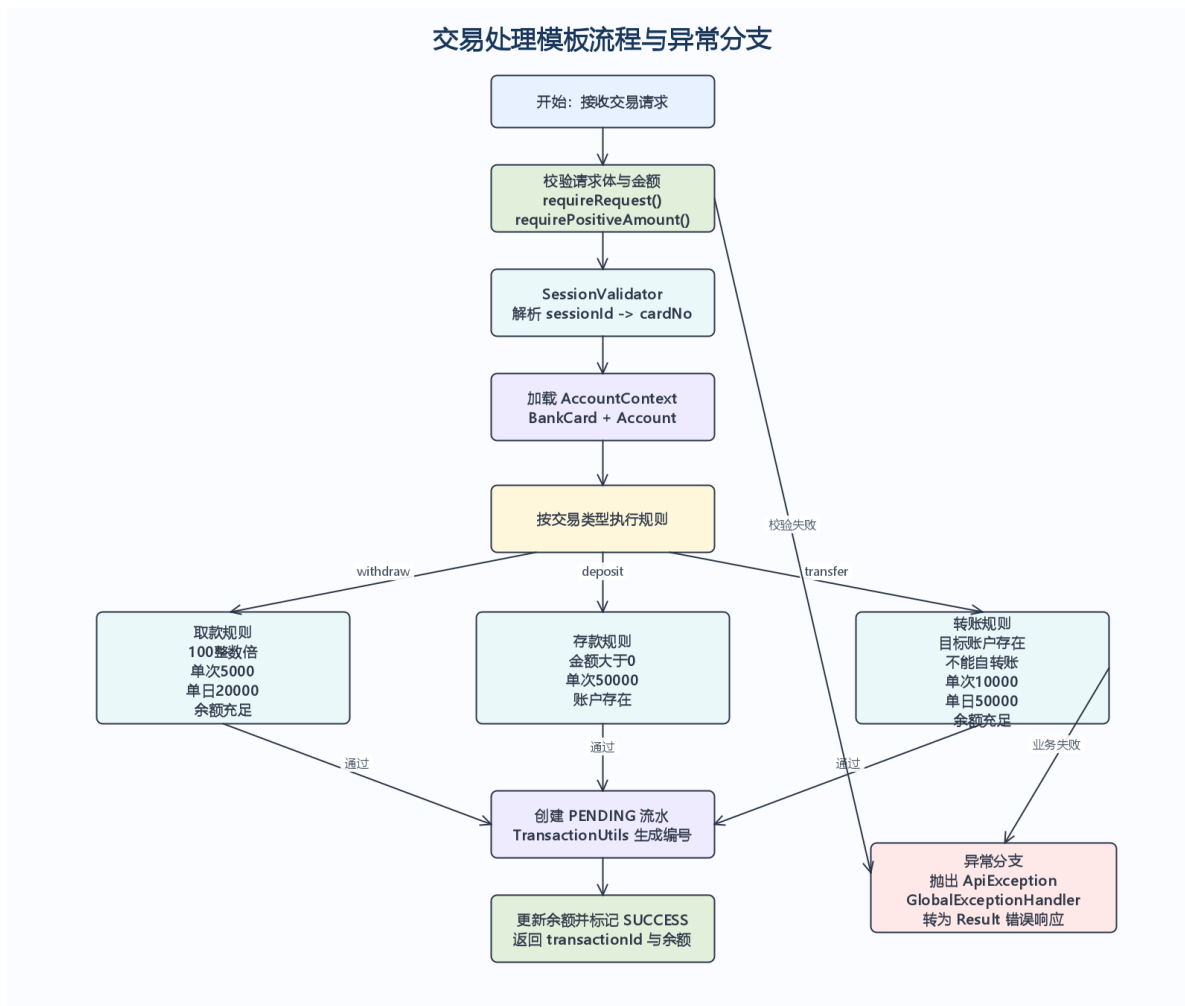


图 3-3 交易处理模板流程与异常分支

图 3-3 将这一公共骨架抽象为模板流程。当前代码并未引入抽象基类来强制实现 Template Method，但已经通过 `requireRequest`、`requirePositiveAmount`、`loadAccountContext`、`createTransaction`、`markSuccess` 等私有公共方法把重复步骤集中起来。这种写法适合当前课程项目规模：既能体现模板方法思想，又不会为了模式而过度拆分。若第三次迭代增加手续费、凭条策略或更多交易类型，可以在此基础上进一步抽取 `TransactionOperation` 或 `TransactionRule` 接口。

3.5 Proxy 模式：事务一致性与 Mapper 代理

转账是本轮最能体现架构风险的场景，因为它同时修改源账户和目标账户，还要记录至少一条对外可查询的交易流水。如果没有明确的事务边界，系统可能出现“源账户已扣款、目标账户未入账”或“余额已变更、流水未记录”等不一致状态。当前实现通过 Spring 的 `@Transactional` 在服务方法外层形成事务代理，业务代码只表达交易步骤，事务提交和异常回滚由框架代理统一处理。

3.6 Adapter 模式：公开契约统一与兼容

第二次迭代除了实现交易功能，还完成了一次重要的公开契约统一：会话字段以 sessionId 为主，账户路径以 /api/atm/accounts/* 为主，交易流水号以 transactionId 为对外标识，转账目标账户字段统一为 targetAccountNo。这类统一如果直接扩散到所有 Controller 和 Service，很容易让业务层同时处理新老字段，从而造成后续文档、前端和测试示例持续漂移。

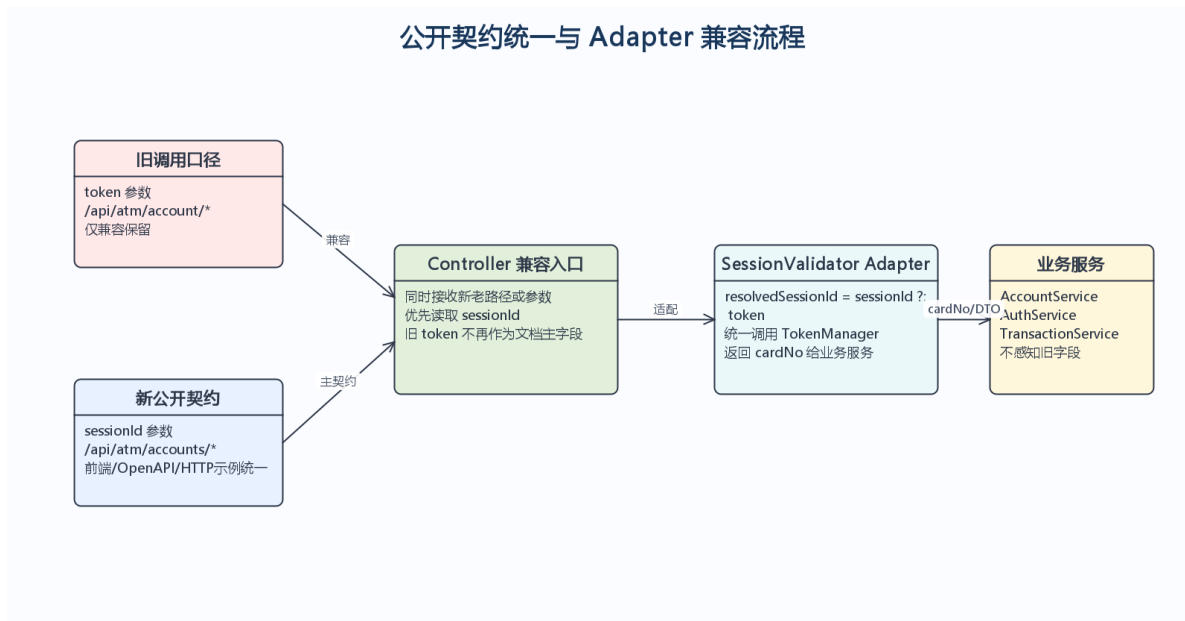


图 3-5 公开契约统一与 Adapter 兼容流程

图 3-5 展示了当前的适配思路。对外文档和前端调用统一使用 sessionId，但后端仍在少数兼容入口保留 token 作为旧别名。SessionValidator 负责将 sessionId 或 token 解析为统一的 resolvedSessionId，再调用 TokenManager 获取 cardNo。这样一来，AccountService、AuthService 和 TransactionService 都只面对统一后的业务身份，不必知道请求方使用的是新字段还是旧字段。

契约项	第二次迭代主口径	兼容与边界说明
会话字段	sessionId	token 仅作为旧请求兼容别名，不再作为新文档主字段
账户路径	/api/atm/accounts/*	/api/atm/account/* 仅作为后端兼容路径保留
交易流水号	transactionId	对外返回和交易详情查询均使用该字段
转账目标账户	targetAccountNo	targetBank 只保留在实体和数据库层用于后续跨行扩展
统一响应结构	code、message、data、timestamp	前端 mock、OpenAPI、HTTP 示例与后端 Result 保持一致

3.7 操作契约与关键业务规则

在第一章和第二章中，操作契约已经用于说明系统操作的前置条件、后置条件和异常分支。第二次迭代后，交易类操作从“骨架接口返回 501”升级为真实状态变更操作，因此需要重新明确关键操作契约。本节不再重复所有字段定义，而是聚焦会导致账户状态变化或会影响安全边界的操作。

系统操作	前置条件	后置条件	异常/替代结果
withdraw(sessionId, amount, printReceipt)	sessionId 有效；金额大于 0；账户存在且余额足够	生成 transactionId；扣减账户余额；交易状态为 SUCCESS；返回 remainingBalance	金额非 100 整数倍、超单次或单日限额、余额不足时返回 400，不产生错误扣款
deposit(sessionId, amount, printReceipt)	sessionId 有效；金额大于 0；账户存在	增加账户余额；写入交易流水；返回 updatedBalance	超单次存款限额或账户不存在时拒绝交易
transfer(sessionId, targetAccountNo, amount, printReceipt)	sessionId 有效；目标账户存在；不能自转账；余额足够	源账户扣款、目标账户入账；写入转出和转入流水；返回 remainingBalance	目标账户不存在返回 404；余额不足、超限额或自转账返回 400；事务失败时整体回滚
changePassword(sessionId, oldPassword, newPassword)	sessionId 有效；旧密码正确；新密码符合规则	更新银行卡密码；当前 sessionId 失效，需要重新登录	旧密码错误、会话无效或密码格式不合规时返回失败
getTransactionById(transactionId)	transactionId 非空	返回交易类型、金额、交易状态、交易前后余额、目标账户等详情	交易编号不存在返回 404

这些契约与第二次迭代记录中的公开字段保持一致：取款、存款请求由 sessionId、amount、printReceipt 组成；转账请求额外包含 targetAccountNo；交易详情以 path 参数 transactionId 查询。业务规则也与交易模块设计文档保持一致，只是当前实现进一步补充了事务注解、每日累计金额查询和目标账户校验的实际代码路径。

3.8 前后端联调与验证

第二次迭代的前端不再只提供占位入口，而是形成了可演示的核心交易页面。WithdrawView、DepositView、TransferView 和 ChangePasswordView 通过统一 API 封装调用后端接口，并在成功后更新页面状态和 Pinia 中的余额。mock 模式仍用于无后端环境下演示，但真实后端已经支持取款、存款、转账、修改密码和交易详情，因而本轮可同时覆盖前端演示和真实联调两个场景。

验证项	验证方式	预期结果
后端自动化测试	在 atm-server-auth 目录执行 <code>.\mvnw.cmd test</code>	Tests run: 13, Failures: 0, Errors: 0, Skipped: 0, BUILD SUCCESS
手工接口链路	按 <code>api-test.http</code> 先登录获取 sessionId，再依次调用账户、取	每个成功交易返回 transactionId，交易详情可查

	款、存款、转账、交易详情接口	询，异常示例返回明确错误信息
前端真实联调	使用 npm run dev 或 npm run dev:real 连接 http://localhost:8080	主菜单进入取款、存款、转账、修改密码页面后可调用真实后端
前端 mock 演示	使用 npm run dev:mock 打开 mock 页面	无后端时仍可演示第二次迭代业务流程和错误提示
契约一致性检查	比对 openapi-atm.yaml、README、FUNCTIONS、api-test.http 和前端 atm.js	字段统一为 sessionId、transactionId、targetAccountNo，账户路径统一为 /api/atm/accounts/*

从验收角度看，本章关注的重点不是单纯“接口能访问”，而是“模型、代码、接口文档和验证材料是否说明同一件事”。例如，第一次迭代中交易接口返回 501 是符合骨架边界的；第二次迭代后，交易接口必须真正完成金额规则校验、账户余额变更和交易详情查询。这种分阶段验收方式能让报告内容与项目实际状态保持一致。

3.9 亮点、风险与拓展点

第二次迭代的亮点不只在于补齐了取款、存款和转账页面，更在于把交易功能放入相对稳定的设计结构中：前端只关心页面输入和结果展示；Controller 提供稳定接口；TransactionService 隐藏复杂交易流程；SessionValidator 统一身份契约；事务代理和 Mapper 代理承担基础设施职责。这种结构使后续迭代可以在不频繁改变公开接口的前提下继续增加凭条、流水列表、设备状态和异常恢复能力。

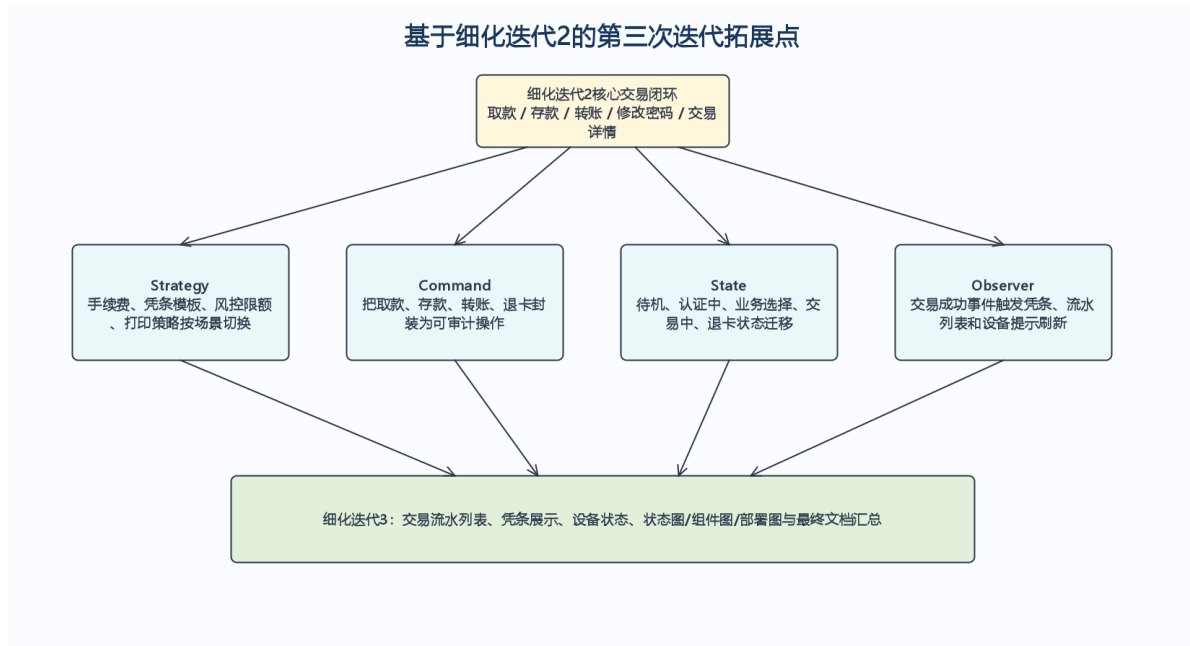


图 3-6 基于细化迭代 2 的第三次迭代拓展点

类别	内容	说明
----	----	----

亮点 1：交易一致性	转账在同一事务中完成源账户扣款、目标账户入账和交易流水写入	直接回应 ATM 系统中账务一致性风险，也是本轮最核心的后端设计价值
亮点 2：契约统一	sessionId、transactionId、targetAccountNo 和 /api/atm/accounts/* 形成统一公开口径	减少前端、后端 DTO、OpenAPI 和测试示例之间的字段漂移
亮点 3：模式意识	以 Facade、Proxy、Adapter 和 Template Method 思想组织交易实现	不是为了堆砌模式，而是用模式解释边界封装、事务代理、兼容适配和公共流程复用
风险 1：规则分支继续增长	取款、存款、转账规则目前仍集中在 TransactionServiceImpl 中	后续如增加手续费、跨行转账、设备现金校验，建议抽取策略接口
风险 2：设备和凭条尚未实现	printReceipt 当前是请求字段和后续衔接点，未形成真实打印或凭条查询闭环	应在第三次迭代补充凭条页面、凭条接口和打印失败处理
拓展点：第三次迭代模式	Strategy、Command、State、Observer 可分别服务于规则策略、ATM 操作封装、会话状态和交易事件扩展	与总体计划中第四章状态机建模和最终文档汇总相衔接

3.10 本章小结

本章依据总体迭代计划中“细化迭代 2 - Applying GoF Design Patterns”的要求，在第二次迭代实现结果基础上说明了 ATM 系统核心交易能力的设计模式应用。从功能层面看，系统已经完成取款、存款、转账、修改密码、完整账户信息、交易前校验和交易详情查询，核心业务闭环已经从第一次迭代的骨架状态推进到可验证状态。从设计层面看，TransactionService 作为服务门面封装了复杂交易流程，Spring 和 MyBatis 代理隐藏了事务与数据访问细节，SessionValidator 作为适配层完成新老会话字段统一，交易服务内部则以模板方法思想复用公共流程并保留交易类型差异。

同时，本章也明确了第二次迭代的边界：交易流水列表、凭条展示、真实打印、ATM 设备状态和更完整的状态机建模仍属于后续细化迭代 3 的主要任务。因此，下一章可以在本章形成的交易闭环和模式应用基础上继续展开 UML State Machine Diagrams and Modeling，并对前述模型进行 refine，最终形成完整的课程实验文档。